

THE MORPHISMS OF DIRECTED CONTAINERS ARE COFUNCTORS, NOT FUNCTORS

MACBETH

ABSTRACT. Directed containers are equivalent, at the level of objects, to small categories: this dictionary underlies their use as a syntax for composition in supply chains, agent orchestration, and bidirectional transformation. It is tempting to extend the dictionary to morphisms by declaring that a morphism of directed containers is a functor. We show this is false on grounds of variance. The position component of a container morphism pulls positions *backward*, $f_s^\sharp: P'(fs) \rightarrow P(s)$, whereas a functor pushes morphisms *forward*; the two variances are opposite. Writing out the compatibility conditions of a *directed*-container morphism, we find they are exactly the cofunctor (retrofunctor) laws on the associated categories. The resulting assignment is an isomorphism of categories $\mathbf{DCont} \cong \mathbf{Cof}$, where $\mathbf{Cof}$ is the category of small categories and cofunctors — not $\mathbf{DCont} \simeq \mathbf{Cat}$, which fails already on a one-object source. The opposite (covariant) variance recovers functors exactly, so functors and cofunctors are the two variances of a single notion of structured map. This is not a defect but the point: the contravariant half is precisely the *Put* of a delta lens, so a directed-container morphism is the update-propagating component of a bidirectional transformation. All counts are verified combinatorially over 36 ordered pairs of small categories.

1. INTRODUCTION

A directed container [2, 3] is a container $S \triangleleft P$ equipped with the data of an indexed monoid: a root $\mathbf{o}(s) \in P(s)$, a sub-shape operation $s \downarrow p$, and a shift $p \oplus q$ that compose positions associatively with the root as unit. The motivating theorem is that this data is *exactly* the data of a small category: shapes are objects, positions out of s are morphisms out of s , the root is the identity, the shift is composition, and $s \downarrow p$ is the codomain of p [3]. This object-level dictionary

directed containers $\cong$ small categories

is what lets one read a category as a coalgebraic, position-based structure — the form in which composition is implemented in applications: a supply chain as composable sub-shapes, an agent hierarchy as composable positions, a state space as composable updates.

A dictionary on objects invites a dictionary on morphisms. The roadmap slogan — and a natural first guess — is that a morphism of directed containers should be a *functor* of the associated categories, upgrading the bijection to an equivalence $\mathbf{DCont} \simeq \mathbf{Cat}$. This note shows the slogan is wrong, identifies the correct statement, and argues that the correction is exactly what makes the bidirectional applications work.

The variance problem. A morphism of containers $(S \triangleleft P) \rightarrow (S' \triangleleft P')$ is a pair $(f, f^\sharp)$ with $f: S \rightarrow S'$ *forward* on shapes but $f_s^\sharp: P'(fs) \rightarrow P(s)$ *backward* on positions: it takes a position out of the target shape fs and returns a position out of the source shape s . Under the dictionary $f_s^\sharp$ sends a morphism out of fs to a morphism out of s . A functor, by contrast, sends a morphism out of s to a morphism out of Fs , a map $P(s) \rightarrow P'(Fs)$ running the *other way*. The variances are opposite, so “a directed-container morphism is a functor” is not merely unproved — it is ill-typed.

The result. The maps with this backward variance, subject to the directed structure, have a name. A *cofunctor* (retrofunctor) [1, 4] $\varphi: \mathcal{C} \leftarrow \mathcal{D}$ is an object map together with a lifting operation that takes an object c and a morphism out of $\varphi_0 c$ and returns a morphism out of c , subject to three laws. Writing the compatibility conditions (M0)–(M2) of a directed-container

morphism and reading them through the dictionary, we find they *are* the cofunctor laws (C0)–(C2), term for term. The assignment is a bijection on every hom-set and respects composition, and it is a bijection on objects; hence:

$$\boxed{\mathbf{DCont} \cong \mathbf{Cof}}$$

an isomorphism of categories, where $\mathbf{Cof}$ is the category of small categories and cofunctors (Theorem 3.5). The would-be statement $\mathbf{DCont} \simeq \mathbf{Cat}$ is false: already over a one-object source there are strictly fewer cofunctors than functors (Proposition 4.2), so no full functor $\mathbf{DCont} \rightarrow \mathbf{Cat}$ can agree with the object dictionary.

The method, and why it is not a defect. The proof is a translation: the data $(f, f^\#)$ and a cofunctor $(\varphi_0, \varphi^\dagger)$ are literally the same data, and (M0)–(M2) match (C0)–(C2) under the substitution “position out of s = morphism out of s ” and “ $\oplus$ = reverse composition”. The covariant variant — a position map $P(s) \rightarrow P'(fs)$ subject to the mirror-image conditions — recovers functors exactly (Proposition 4.1). So a directed structure carries *two* natural notions of map, of opposite variance: covariant gives functors, contravariant gives cofunctors, and only the latter is a morphism of the underlying container.

This is the useful half. A *delta lens* [6, 4, 5] — the categorical model of a bidirectional transformation — is precisely a *Get* functor together with a *Put* cofunctor sharing an object map, related by a section condition. Under $\mathbf{DCont} \cong \mathbf{Cof}$, the *Put* cofunctor *is* a directed-container morphism. So the contravariant variance flagged as a problem above is exactly the component that propagates an update from a view back to its source (Corollary 5.3). Reading a directed-container map as a functor would silently discard the update semantics — and, as we show, even change the count of maps.

Outline. Section 2 recalls directed containers, the object dictionary, and cofunctors. Section 3 writes the morphism conditions and proves $\mathbf{DCont} \cong \mathbf{Cof}$. Section 4 places functors as the opposite variance and shows the dictionary does not extend to $\mathbf{Cat}$. Section 5 identifies a directed-container morphism with the *Put* of a delta lens. Section 6 reports the combinatorial verification. Section 7 places the result among its 2026 neighbours, and Section 8 discusses consequences and what remains.

2. DIRECTED CONTAINERS AND COFUNCTORS

We recall only what the proof uses, following [2, 3] for containers and [1, 4] for cofunctors.

2.1. Directed containers and the object dictionary. A *container* $S \triangleleft P$ is a set S of *shapes* together with a family $P: S \rightarrow \mathbf{Set}$ of *positions*. A *directed container* structure on $S \triangleleft P$ is a root $\mathbf{o}(s) \in P(s)$, a sub-shape map $s \downarrow p \in S$ for $p \in P(s)$, and a shift $p \oplus q \in P(s)$ for $p \in P(s)$, $q \in P(s \downarrow p)$, satisfying the indexed-monoid laws (root is a two-sided unit for $\oplus$; $\oplus$ is associative; $\downarrow$ acts compatibly). We refer to [3] for the precise axioms; we will use them only through the dictionary below.

Theorem 2.1 (Object dictionary [3]). *Directed-container structures on $S \triangleleft P$ are in bijection with small categories $\mathcal{C}$ with object set S and, for each s , $P(s) = \coprod_{s'} \mathcal{C}(s, s')$ the set of morphisms out of s , under*

$$\mathbf{ob} \mathcal{C} = S, \quad \mathcal{C}(s, s') = \{p \in P(s) : s \downarrow p = s'\}, \quad \mathbf{id}_s = \mathbf{o}(s), \quad q \circ p = p \oplus q.$$

In particular $s \downarrow p = \mathbf{cod}(p)$, and every small category arises from a unique directed container.

Throughout, $\mathcal{C}(D)$ denotes the category of a directed container D . We will move freely across the dictionary: a position $p \in P(s)$ *is* a morphism out of s , with codomain $s \downarrow p$; the root *is* the identity; and $\oplus$ is composition in reverse order, $p \oplus q = q \circ p$.

2.2. Cofunctors.

Definition 2.2 (Cofunctor [1, 4]). A *cofunctor* $\varphi: \mathcal{C} \leftarrow \mathcal{D}$ between small categories consists of

- an object map $\varphi_0: \mathbf{ob} \mathcal{C} \rightarrow \mathbf{ob} \mathcal{D}$;

- a *lift*: for each object c of $\mathcal{C}$ and each morphism $h: \varphi_0 c \rightarrow d$ of $\mathcal{D}$, a morphism $\varphi^\uparrow(c, h): c \rightarrow c'$ of $\mathcal{C}$,

subject to:

$$\begin{aligned}
(\text{C0}) \quad & \varphi_0(\text{cod} \varphi^\uparrow(c, h)) = \text{cod}(h) && (\text{codomain}) \\
(\text{C1}) \quad & \varphi^\uparrow(c, \text{id}_{\varphi_0 c}) = \text{id}_c && (\text{unit}) \\
(\text{C2}) \quad & \varphi^\uparrow(c, h' \circ h) = \varphi^\uparrow(c', h') \circ \varphi^\uparrow(c, h), \quad c' = \text{cod} \varphi^\uparrow(c, h) && (\text{composition})
\end{aligned}$$

In (C2), $h: \varphi_0 c \rightarrow d$ and $h': d \rightarrow d''$; the term $\varphi^\uparrow(c', h')$ typechecks because $\varphi_0 c' = d = \text{dom}(h')$ by (C0). Cofunctors compose — object maps forward, lifts backward, $(\psi\varphi)^\uparrow(c, m) = \varphi^\uparrow(c, \psi^\uparrow(\varphi_0 c, m))$ — with identities $(\text{id}, \text{id}^\uparrow)$, forming a category **Cof**.

A cofunctor is *not* a contravariant functor [4, p. 188]: it does not act on all of $\mathcal{C}$'s morphisms, only lifting morphisms presented at the φ_0 -image of an object. Equivalently [4], a cofunctor is a span $\mathcal{C} \xleftarrow{\varphi} X \xrightarrow{\bar{\varphi}} \mathcal{D}$ with φ bijective-on-objects and $\bar{\varphi}$ a discrete opfibration; cofunctors are the comonoid morphisms in $(\mathbf{Poly}, \triangleleft)$ [3, 9]. We will use only the elementary form of Definition 2.2.

3. THE MORPHISM DICTIONARY: $\mathbf{DCont} \cong \mathbf{Cof}$

We now write the conditions that make a container morphism a morphism of *directed* containers, and read them as the cofunctor laws.

Definition 3.1 (Directed-container morphism). A *morphism of directed containers* $D \rightarrow D'$ is a container morphism $(f, f^\sharp)$, $f: S \rightarrow S'$ and $f_s^\sharp: P'(fs) \rightarrow P(s)$, satisfying for all $s \in S$, $h \in P'(fs)$ and $k \in P'((fs) \downarrow' h)$:

$$\begin{aligned}
(\text{M0}) \quad & f(s \downarrow f_s^\sharp(h)) = (fs) \downarrow' h && (\downarrow\text{-compatibility}) \\
(\text{M1}) \quad & f_s^\sharp(o'(fs)) = o(s) && (\text{root}) \\
(\text{M2}) \quad & f_s^\sharp(h \oplus' k) = f_s^\sharp(h) \oplus f_{s \downarrow f_s^\sharp(h)}^\sharp(k) && (\text{shift homomorphism})
\end{aligned}$$

Remark 3.2 (Why (M2) typechecks). Put $p := f_s^\sharp(h) \in P(s)$. The left side applies $f_s^\sharp$ to $h \oplus' k \in P'(fs)$. On the right, (M0) gives $(fs) \downarrow' h = f(s \downarrow p)$, so $k \in P'(f(s \downarrow p))$ and $f_{s \downarrow p}^\sharp(k) \in P(s \downarrow p)$; hence $p \oplus f_{s \downarrow p}^\sharp(k) \in P(s)$ is defined. The shift condition is well-formed precisely *because* of the codomain condition — exactly as (C2) needs (C0).

Lemma 3.3 (Hom-set bijection). Let D, D' be directed containers with categories $\mathcal{C} = \mathcal{C}(D)$, $\mathcal{C}' = \mathcal{C}(D')$. The assignment

$$(f, f^\sharp) \mapsto (\varphi_0 := f, \varphi^\uparrow(s, h) := f_s^\sharp(h))$$

is a bijection between morphisms $D \rightarrow D'$ of directed containers (Definition 3.1) and cofunctors $\mathcal{C} \leftarrow \mathcal{C}'$ (Definition 2.2).

Proof. The key observation is that there is nothing to construct: under the dictionary the two sides are the *same data*. A position $h \in P'(fs)$ is a morphism out of fs in $\mathcal{C}'$ with codomain $(fs) \downarrow' h$, and $f_s^\sharp(h) \in P(s)$ is a morphism out of s in $\mathcal{C}$ with codomain $s \downarrow f_s^\sharp(h)$. So f is an object map and $f^\sharp$ is a lift; what must be checked is that the conditions correspond. We match them term by term.

$(\text{M0}) \Leftrightarrow (\text{C0})$. (M0) reads $f(s \downarrow f_s^\sharp(h)) = (fs) \downarrow' h$. The left side is $\varphi_0(\text{cod} \varphi^\uparrow(s, h))$; the right side is $\text{cod}(h)$ in $\mathcal{C}'$. This is (C0).

$(\text{M1}) \Leftrightarrow (\text{C1})$. The identity at fs is $o'(fs) = \text{id}_{fs}$ and the identity at s is $o(s) = \text{id}_s$, so $f_s^\sharp(o'(fs)) = o(s)$ is $\varphi^\uparrow(s, \text{id}_{\varphi_0 s}) = \text{id}_s$. This is (C1).

$(\text{M2}) \Leftrightarrow (\text{C2})$. Translate $\oplus$ into composition by $x \oplus y = y \circ x$. With $p = f_s^\sharp(h)$ and $s' = s \downarrow p$, the right side of (M2) is

$$p \oplus f_{s'}^\sharp(k) = f_{s'}^\sharp(k) \circ f_s^\sharp(h) = \varphi^\uparrow(s', k) \circ \varphi^\uparrow(s, h),$$

and the left side is $f_s^\sharp(h \oplus' k) = f_s^\sharp(k \circ' h) = \varphi^\uparrow(s, k \circ' h) = \varphi^\uparrow(s', k) \circ \varphi^\uparrow(s, h)$ with $s' = \text{cod} \varphi^\uparrow(s, h)$, which is (C2).

The three conditions correspond under the identity map on data, so the assignment is a bijection on hom-sets, with inverse the same identification read backward. $\square$

Lemma 3.4 (Functoriality). $\mathcal{C}(-)$ with the assignment of Lemma 3.3 is a functor $\Phi: \mathbf{DCont} \rightarrow \mathbf{Cof}$.

Proof. The identity container morphism (id_S, id) maps to $(\text{id}, \text{id}^\uparrow)$, the identity cofunctor. For composition, let $(f, f^\sharp): D \rightarrow D'$ and $(g, g^\sharp): D' \rightarrow D''$. The container composite has object map $g \circ f$ and position map

$$(g \circ f)_s^\sharp = f_s^\sharp \circ g_{f_s}^\sharp: P''(gfs) \xrightarrow{g_{f_s}^\sharp} P'(fs) \xrightarrow{f_s^\sharp} P(s),$$

positions pulling back through both stages. Under Φ this is the cofunctor with object map $g_0 \circ f_0$ and lift $m \mapsto f_s^\sharp(g_{f_s}^\sharp(m)) = \varphi^\uparrow(s, \psi^\uparrow(\varphi_0 s, m))$, which is the composite cofunctor $\psi\varphi$ of Definition 2.2. So Φ preserves composition. $\square$

Theorem 3.5 (The morphism dictionary). $\Phi: \mathbf{DCont} \rightarrow \mathbf{Cof}$ is an isomorphism of categories, $\mathbf{DCont} \cong \mathbf{Cof}$.

Proof. Φ is fully faithful by Lemma 3.3 (a bijection on every hom-set) and a functor by Lemma 3.4. On objects $D \mapsto \mathcal{C}(D)$ is a bijection by Theorem 2.1. A bijective-on-objects, fully faithful functor is an isomorphism of categories: the object bijection and the hom bijections assemble into a strict two-sided inverse. $\square$

4. FUNCTORS ARE THE OPPOSITE VARIANCE

Theorem 3.5 says the maps native to directed containers are cofunctors. Where, then, do functors sit? At the opposite variance — and *not* as a refinement of the same dictionary.

Proposition 4.1 (Functors = covariant-position morphisms). *Call an op-variance morphism $D \rightarrow D'$ a pair (f, ϕ) with $f: S \rightarrow S'$ and a covariant position map $\phi_s: P(s) \rightarrow P'(fs)$ satisfying (N0) $f(s \downarrow p) = (fs) \downarrow' \phi_s(p)$, (N1) $\phi_s(o(s)) = o'(fs)$, and (N2) $\phi_s(p \oplus q) = \phi_s(p) \oplus' \phi_{s \downarrow p}(q)$. Then $(f, \phi) \mapsto F$, with $F = f$ on objects and $F(p) = \phi_s(p)$ on a morphism p out of s , is a bijection onto functors $\mathcal{C}(D) \rightarrow \mathcal{C}(D')$.*

Proof. A functor is exactly an object map f , a map of out-sets $\phi_s = F|_{P(s)}: P(s) \rightarrow P'(fs)$ for each s , preserving codomains (N0), identities (N1), and composition (N2: $F(q \circ p) = F(q) \circ F(p)$, i.e. $F(p \oplus q) = \phi_s(p) \oplus' \phi_{s \downarrow p}(q)$). Each datum and law matches under the identity on data. $\square$

So a directed structure carries two notions of structured map, of opposite variance:

$$\begin{aligned} \text{covariant positions } P(s) &\rightarrow P'(fs) \rightsquigarrow \mathbf{functors}, \\ \text{contravariant positions } P'(fs) &\rightarrow P(s) \rightsquigarrow \mathbf{cofunctors}, \end{aligned}$$

and only the contravariant one is a morphism of the underlying container. This is not a matter of which is “better”; it is that the container forgets to the second.

The two are genuinely different, even in cardinality.

Proposition 4.2 (The dictionary does not extend to $\mathbf{Cat}$). *Let $\Phi_0: \text{ob } \mathbf{DCont} \rightarrow \text{ob } \mathbf{Cat}$, $D \mapsto \mathcal{C}(D)$, be the object dictionary of Theorem 2.1. There is no functor $\mathbf{DCont} \rightarrow \mathbf{Cat}$ that agrees with Φ_0 on objects and is full. In particular the object dictionary does not extend to an equivalence $\mathbf{DCont} \simeq \mathbf{Cat}$.*

Proof. Let D_1, D_2 be the directed containers of the terminal category $\mathbf{1}$ and of $\mathbf{2} = \{0 \xrightarrow{u} 1\}$. By Theorem 3.5, $|\mathbf{DCont}(D_1, D_2)| = |\mathbf{Cof}(\mathbf{1}, \mathbf{2})|$, and we count the latter directly. A cofunctor $\mathbf{1} \leftarrow \mathbf{2}$ picks $\varphi_0(*) \in \{0, 1\}$ and must lift every morphism out of $\varphi_0(*)$ to the unique morphism id_* out of $*$, with (C0) forcing $\varphi_0(\text{cod}(\text{lift})) = \text{cod}$. Choosing $\varphi_0(*) = 0$ fails: $u: 0 \rightarrow 1$ would lift to id_* , but then (C0) demands $\varphi_0(*) = \text{cod}(u) = 1 \neq 0$. Choosing $\varphi_0(*) = 1$ works: the

only morphism out of 1 is id_1 , lifting to id_* . So $|\mathbf{Cof}(1, 2)| = 1$, whence $|\mathbf{DCont}(D_1, D_2)| = 1$. But $|\mathbf{Cat}(1, 2)| = 2$ (a functor $1 \rightarrow 2$ chooses either object). A functor $G: \mathbf{DCont} \rightarrow \mathbf{Cat}$ equal to Φ_0 on objects would restrict to a map from a 1-element hom-set onto a 2-element hom-set, which cannot be surjective; so G is not full. $\square$

Remark 4.3 (**Cof** and **Cat** are incomparable, not nested). The failure is not that **Cof** sits inside **Cat** with fewer maps. The comparison is not even monotone over common objects: for the 3-chain **3**, $|\mathbf{Cat}(3, 2)| = 4 < 5 = |\mathbf{Cof}(3, 2)|$, while $|\mathbf{Cat}(2, 2)| = 3 > 2 = |\mathbf{Cof}(2, 2)|$ (Table 1). Neither hom-set contains the other. In **Poly** this is the statement that functors and cofunctors occupy different hom-objects: cofunctors are the comonoid morphisms in $(\mathbf{Poly}, \triangleleft)$, whereas functors are carried by bicomodules [3, 9]. We do not claim a separation of **Cof** and **Cat** as abstract categories; Proposition 4.2 settles the question the dictionary actually poses.

5. THE COFUNCTOR IS THE PUT OF A DELTA LENS

The contravariant variance is the useful one. A *delta lens* [6, 4] is the categorical model of a bidirectional transformation between a source category A and a view category B : it propagates a chosen update on the view back to a coherent update on the source.

Definition 5.1 (Delta lens [5]). A *delta lens* $(f, \varphi): A \rightrightarrows B$ consists of a functor $f: A \rightarrow B$ (the *Get*) and, for each object $a \in A$ and each morphism $u: fa \rightarrow b$ in B , a morphism $\varphi(a, u): a \rightarrow a'$ in A (the *Put/lift*), satisfying

$$\begin{aligned} (\text{PutGet}) \quad & f \cdot \varphi(a, u) = u, \\ (\text{PutId}) \quad & \varphi(a, \text{id}_{fa}) = \text{id}_a, \\ (\text{PutPut}) \quad & \varphi(a, v \circ u) = \varphi(a', v) \circ \varphi(a, u), \quad a' = \text{cod} \varphi(a, u). \end{aligned}$$

The Put data φ alone — an object map (here f on objects) plus a lift satisfying the codomain, unit and composition laws — is exactly a cofunctor $A \leftarrow B$ [5, Def. 3] (object map $f_0: A_0 \rightarrow B_0$, lifts landing in the source A , matching the convention of Definition 2.2). The Get f supplies the functor; (PutGet) is the *section condition* relating the two, asserting that applying Get to a lifted update returns the original update u . Clarke makes the decomposition precise: an (internal) lens is a functor together with a cofunctor on a *shared object map*, related by (PutGet); equivalently a commuting triangle $A \xleftarrow{\varphi} \Lambda \xrightarrow{\bar{\varphi}} B$ with φ identity-on-objects and $\bar{\varphi}$ a discrete opfibration, and $f \circ \varphi = \bar{\varphi}$ [4, Cor. 20]. Indeed $f: \mathbf{DLens}(B) \rightarrow \mathbf{Cof}(B)$ forgetting the Get is comonadic [5, Thm. 9].

Remark 5.2 (A correction worth flagging). The section condition is (PutGet), $f \cdot \varphi(a, u) = u$, not “ $f \circ \varphi = \text{id}$ ”. Applying Get to a lifted update returns the original update, not an identity; the latter holds only when read as a section of the fibre of updates over each object. We state this because the loose form recurs.

Combining Theorem 3.5 with the decomposition gives the application hook.

Corollary 5.3 (A directed-container morphism is the Put of a lens). *Under the isomorphism $\mathbf{DCont} \cong \mathbf{Cof}$ of Theorem 3.5, the Put cofunctor of a delta lens $(f, \varphi): A \rightrightarrows B$ corresponds to a morphism of directed containers $D_A \rightarrow D_B$ between the directed containers of A and B . The covariant component recovered by Proposition 4.1 is the Get functor. Hence a delta lens is a directed-container morphism (the Put) together with a functor on the same shapes (the Get), tied by (PutGet).*

Proof. Immediate from Theorem 3.5 and Definition 5.1: the Put φ is a cofunctor $A \leftarrow B$ [5], which Φ^{-1} carries to a directed-container morphism $D_A \rightarrow D_B$; Proposition 4.1 identifies the Get with the covariant morphism on the same object map. $\square$

This is why the variance matters in applications. Whenever a construction is transported along the equivalence chain — composing supply-chain stages, propagating a change through an agent hierarchy, updating a derived view of a state space — the maps that carry update

semantics are the cofunctors, i.e. the directed-container morphisms. Reading them as functors keeps the Get and discards the Put.

6. COMPUTATIONAL VERIFICATION

We verified the correspondence combinatorially. A script represents a small category concretely and enumerates, between two categories C, D : functors $C \rightarrow D$; cofunctors $C \leftarrow D$ via (C0)–(C2); directed-container morphisms via (M0)–(M2) computed *directly* from the container data ($\circ, \downarrow, \oplus$); and op-variance morphisms via (N0)–(N2). Over all 36 ordered pairs from $\{\mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbb{Z}/2, \mathbb{Z}/3, M\}$, with M the two-element idempotent monoid $\{1, t\}$, $t^2 = t$:

$$\#\{\text{DCont-morphisms}\} = \#\{\text{cofunctors}\} \quad \text{in all 36 cases,}$$

$$\#\{\text{op-variance morphisms}\} = \#\{\text{functors}\} \quad \text{in all 36 cases,}$$

confirming Lemma 3.3 and Proposition 4.1; while $\#\{\text{functors}\} \neq \#\{\text{cofunctors}\}$ in 17 of the 36 cases, confirming Proposition 4.2. A representative excerpt:

$C \rightarrow D$	$\#\text{functors}$	$\#\text{cofunctors}$	$\#\text{DCont-morphisms}$
$\mathbf{1} \rightarrow \mathbf{2}$	2	1	1
$\mathbf{2} \rightarrow \mathbf{2}$	3	2	2
$\mathbf{2} \rightarrow \mathbb{Z}/3$	3	1	1
$\mathbf{3} \rightarrow \mathbf{2}$	4	5	5
$\mathbf{3} \rightarrow \mathbf{3}$	10	6	6
$\mathbb{Z}/3 \rightarrow \mathbb{Z}/3$	3	3	3
$\mathbf{3} \rightarrow M$	4	5	5

TABLE 1. Functors vs. cofunctors vs. directed-container morphisms. $\#\text{cofunctors} = \#\text{DCont-morphisms}$ throughout; $\#\text{functors}$ differs. The rows $\mathbf{3} \rightarrow \mathbf{2}$ and $\mathbf{2} \rightarrow \mathbf{2}$ show that neither of **Cat**, **Cof** dominates the other.

7. RELATED WORK

The object dictionary of Theorem 2.1 and the retrofunctor reading of its morphisms are, by 2026, common infrastructure. Two recent papers build over that infrastructure in directions orthogonal to ours, and locate the present note among its neighbours without displacing it. Both take the equivalence between retrofunctors and polynomial comonoids [3, 4] as *black-box* prior art — exactly the identification that Theorem 3.5 unpacks at the level of container data.

Garner, Renata, and Wu [7] construct a contravariant idempotent adjunction between ranked monads on **Set** and internal categories-with-retrofunctors in the category of locales; its fixed points are the hyperaffine-unary monads on one side and the ample localic categories on the other (their “Stone duality for monads”). Their starting observation is ours: “the notion of morphism we need is not functors, but rather retrofunctors” [7, §1], and they read a retrofunctor $C \leftarrow D$ as a *simulation of transition systems* — the transition-system avatar of the update-propagating Put isolated in Section 5. Their move is to *internalise the base*: **Set** is replaced by **Loc** (or **Top**), and small categories with retrofunctors become their localic counterpart. Our **DCont** $\cong$ **Cof** is the **Set**-level, detopologised ground case of the object over which their adjunction sits; conversely their construction is a data point — not yet an answer — for the question of what the morphism dictionary becomes over a base other than **Set**.

Fairbanks, Carlson, and Spivak [8] characterise topological spaces as density comonads of diagrams of subsets, explicitly motivated by the Ahman–Uustalu characterisation of categories as the polynomial comonads on **Set**; the resulting double category of continuous maps “recovers the double category of functors and retrofunctors of Clarke and Di Meglio” — the same functor/cofunctor pairing that Corollary 5.3 reads as Get and Put. They cite Ahman–Uustalu

as settled prior art and do not touch the directed-container axioms. It is the closest neighbour in vocabulary, and supplies context rather than a competing statement of the morphism dictionary.

8. CONCLUSION AND OUTLOOK

The object dictionary — directed containers are small categories — extends to morphisms, but not in the obvious way. A morphism of directed containers is a *cofunctor*, not a functor, because its position component runs contravariantly; the precise statement is an isomorphism of categories $\mathbf{DCont} \cong \mathbf{Cof}$. Functors are the mirror-image, covariant, notion, and are not morphisms of containers; the dictionary does not extend to $\mathbf{Cat}$, failing already on a one-object source. The contravariant variance is exactly the Put of a delta lens, so a directed-container morphism is the update-propagating half of a bidirectional transformation.

The practical consequence is a correction to make wherever the chain is used: a “map of directed containers” carries update (Put) semantics and must be composed as a cofunctor. In supply-chain composition and agent orchestration — where a map between two staged structures should propagate a downstream change back upstream — this is the difference between the right morphisms and a silently different set of them, of even a different cardinality.

Two directions remain. First, formalisation: $\mathbf{Cof}$ and Theorem 3.5 are natural targets for a Lean development sitting on the directed-container/category dictionary, which would let the correction be enforced rather than remembered. Second, and more speculatively, the lens reading suggests a measure of how much a map “rewrites” as it propagates an update; whether such a diversity invariant of a directed-container morphism is itself compositional is a question we leave open, as it lies beyond the bookkeeping settled here.

REFERENCES

- [1] M. Aguiar. *Internal categories and quantum groups*. PhD thesis, Cornell University, 1997.
- [2] D. Ahman, J. Chapman, and T. Uustalu. *When is a container a comonad?* Logical Methods in Computer Science **10**(3), 2014.
- [3] D. Ahman and T. Uustalu. *Directed containers as categories*. In *Proc. MSFP 2016*, EPTCS **207**, 2016, pp. 89–98.
- [4] B. Clarke. *Internal lenses as functors and cofunctors*. In *Proc. ACT 2019*, EPTCS **323**, 2020, pp. 183–195. [arXiv:2009.06835](https://arxiv.org/abs/2009.06835).
- [5] B. Clarke. *Delta lenses as coalgebras for a comonad*. [arXiv:2108.00390](https://arxiv.org/abs/2108.00390), 2022.
- [6] M. Johnson and R. Rosebrugh. *Delta lenses and opfibrations*. Electronic Communications of the EASST **57**, 2013.
- [7] R. Garner, A. Renata, and N. Wu. *Stone duality for monads*. In *Proc. MFPS 2026*, preliminary proceedings. [arXiv:2603.25710](https://arxiv.org/abs/2603.25710).
- [8] A. Fairbanks, Carlson, and D. I. Spivak. *Comonads as spaces*. [arXiv:2607.15091](https://arxiv.org/abs/2607.15091), 2026.
- [9] B. Shapiro and D. I. Spivak. [arXiv:2405.13157](https://arxiv.org/abs/2405.13157), 2024; on $\mathbf{Cat}^\sharp$, the category of small categories and cofunctors realised inside $(\mathbf{Poly}, <)$.